PhotonDB

Task Assignment Plan

CodeCrafters Redis Stages · 5 People · 4 Weeks

ENGG 102 · Kathmandu University

Jun 19 – Jul 16, 2026

Person	Role	Owns
Amrit Pant (24)	Tech Lead + Infrastructure	TCP Server · Concurrency · Docker · Replication
Aswin Pandey (20)	Protocol Engineer	RESP Parser & Encoder · Dispatcher · Streams
Ayurva Pradhananga (44)	Core Engine	KV Storage · String Commands · Transactions · WATCH
Divyanshi Mainali (8)	Data Structures	Lists · Sorted Sets · Pub/Sub
Nishan Niraula (16)	Persistence + WOW	RDB · AOF · Temporal Key-Value (GETVER/HISTORY/SNAPSHOT)

Difficulty Key		Notes
EASY	MEDIUM	HARD

102	20	16	21	26	19
Total Stages	Amrit	Aswin	Ayurva	Divyanshi	Nishan

WEEK 1 — Foundation

Jun 19 – Jun 25

Sprint Goal: TCP server live, RESP parser working, redis-cli PING → PONG confirmed by all members.

	Stage / Command	Difficulty	Assigned To	What To Implement
1	**Bind to a port**	EASY	Amrit	*Create TCP socket, setsockopt SO_REUSEADDR, bind to 6379, listen()*
2	**Respond to PING**	EASY	Aswin	*Parse raw bytes; reply +PONG\r\n — first proof RESP works*
3	**Respond to multiple PINGs**	EASY	Aswin	*Keep the connection alive; loop over recv(); handle multiple commands*
4	**Handle concurrent clients**	MEDIUM	Amrit	*Spawn a thread per accepted fd; protect shared state with a mutex*
5	**Implement the ECHO command**	MEDIUM	Aswin	*Dispatcher routes ECHO to handler; reply with Bulk String*
6	**Implement SET & GET**	MEDIUM	Ayurva	*Initialise StorageEngine (unordered_map); wire handlers*
7	**Expiry (PX / EX flags)**	MEDIUM	Ayurva	*Extend Entry with optional expiry; add background checker*

Person	Stages this week	Count
Amrit	*Bind to a port, Handle concurrent clients*	2
Aswin	*Respond to PING, Respond to multiple PINGs, Implement the ECHO command*	3
Ayurva	*Implement SET & GET, Expiry (PX / EX flags)*	2

WEEK 2 — Lists, Core Commands & RDB

Jun 26 – Jul 2

Sprint Goal: Full list API working, INCR/DECR done, RDB can save and reload the store on restart.

	Stage / Command	Difficulty	Assigned To	What To Implement
1	**Create a list (RPUSH)**	EASY	Divyanshi	*New map>; RPUSH key val → LRANGE key 0 -1*
2	**Append an element**	EASY	Divyanshi	*RPUSH appends to tail of deque*
3	**Append multiple elements**	EASY	Divyanshi	*RPUSH accepts varargs; iterate and push each*
4	**List elements (positive idx)**	EASY	Divyanshi	*LRANGE key 0 N — slice the deque*
5	**List elements (negative idx)**	EASY	Divyanshi	*Negative index wraps: idx = size + idx*
6	**Prepend elements (LPUSH)**	EASY	Divyanshi	*push_front on the deque*
7	**Query list length (LLEN)**	EASY	Divyanshi	*Return deque.size() as RESP Integer*
8	**Remove an element (LREM)**	EASY	Divyanshi	*Scan deque and erase matching values up to count*
9	**Remove multiple elements**	EASY	Divyanshi	*LREM count > 1 variant; iterate with direction flag*
10	**Blocking retrieval (BLPOP)**	MEDIUM	Divyanshi	*If list empty: block client thread with condition_variable until push*
11	**Blocking retrieval with timeout**	MEDIUM	Divyanshi	*cv.wait_for(lock, timeout_ms); reply nil if timeout*
12	**The TYPE command**	EASY	Aswin	*Route to type-check handler; returns 'string', 'list', 'zset', etc.*
13	**The INCR command (1/3)**	EASY	Ayurva	*Parse value as integer; increment; store back as string*
14	**The INCR command (2/3)**	EASY	Ayurva	*Return new integer value as RESP Integer*
15	**The INCR command (3/3)**	EASY	Ayurva	*Error if value is not an integer; also implement DECR, INCRBY*
16	**RDB file config**	EASY	Nishan	*Read --dir and --dbfilename flags at startup*
17	**Read a key**	MEDIUM	Nishan	*Parse .rdb binary header, locate a single key entry*
18	**Read a string value**	MEDIUM	Nishan	*Decode length-prefixed string encoding from RDB*
19	**Read multiple keys**	MEDIUM	Nishan	*Loop through all key-value pairs in the RDB hash table section*
20	**Read multiple string values**	MEDIUM	Nishan	*Handle all string encoding types (int-encoded, raw, LZF)*
21	**Read value with expiry**	MEDIUM	Nishan	*Parse 0xFC (ms) or 0xFD (s) expiry opcodes; populate entry.expiry*

Person	Stages this week	Count

Divyanshi	*Create a list (RPUSH), Append an element, Append multiple elements, List elements (positive idx), List elements (negative idx), Prepend elements (LPUSH), Query list length (LLEN), Remove an element (LREM), Remove multiple elements, Blocking retrieval (BLPOP), Blocking retrieval with timeout*	**11**
Aswin	*The TYPE command*	**1**
Ayurva	*The INCR command (1/3), The INCR command (2/3), The INCR command (3/3)*	**3**
Nishan	*RDB file config, Read a key, Read a string value, Read multiple keys, Read multiple string values, Read value with expiry*	**6**

WEEK 3 — Streams, Transactions, AOF & Pub/Sub

Jul 3 – Jul 9

Sprint Goal: Streams, transactions, AOF persistence, and Pub/Sub all functional and tested.

	Stage / Command	Difficulty	Assigned To	What To Implement
1	**Create a stream (XADD)**	MEDIUM	Aswin	*New map>; XADD key id field val*
2	**Validating entry IDs**	EASY	Aswin	*Format must be ms-seq; reject out-of-order IDs with error*
3	**Partially auto-generated IDs**	MEDIUM	Aswin	*XADD key ms-* → auto-increment seq for given ms*
4	**Fully auto-generated IDs**	MEDIUM	Aswin	*XADD key * → use system ms + auto seq*
5	**Query entries (XRANGE)**	MEDIUM	Aswin	*Scan stream vector between start and end IDs inclusive*
6	**Query with - (min)**	EASY	Aswin	*'-' is shorthand for 0-0; start from beginning*
7	**Query with + (max)**	EASY	Aswin	*'+' is shorthand for max uint64; go to end*
8	**Query single stream (XREAD)**	MEDIUM	Aswin	*XREAD COUNT n STREAMS key id — return entries after given id*
9	**Query multiple streams (XREAD)**	MEDIUM	Aswin	*XREAD STREAMS k1 k2 id1 id2 — fan out across streams*
10	**Blocking reads (XREAD BLOCK)**	HARD	Aswin	*Block thread until new entry arrives; use condition_variable*
11	**Blocking reads without timeout**	MEDIUM	Aswin	*BLOCK 0 → block indefinitely; only notify wakes it*
12	**Blocking reads using $**	EASY	Aswin	*'$' means 'only new entries'; record cursor on subscribe*
13	**The MULTI command**	EASY	Ayurva	*Per-client queue tx_queue; set in_multi flag*
14	**The EXEC command**	EASY	Ayurva	*Execute queued commands atomically; return array of results*
15	**Empty transaction**	HARD	Ayurva	*EXEC with empty queue returns empty array *3\r\n; not nil*
16	**Queueing commands**	MEDIUM	Ayurva	*While in_multi: push to queue, reply +QUEUED\r\n*
17	**Executing a transaction**	HARD	Ayurva	*Lock store, run all queued commands, unlock, return all replies*
18	**The DISCARD command**	EASY	Ayurva	*Clear queue; unset in_multi; reply +OK\r\n*
19	**Failures within transactions**	MEDIUM	Ayurva	*Runtime errors inside EXEC don't abort rest of transaction*
20	**Multiple transactions**	MEDIUM	Ayurva	*Each client connection has independent tx state; verify isolation*
21	**Default AOF options**	EASY	Nishan	*aof-use-rdb-preamble yes default; no-appendfsync-on-rewrite no*
22	**AOF options from flags**	EASY	Nishan	*Parse --appendonly and --appendfilename from argv*

	Stage / Command	Difficulty	Assigned To	What To Implement
23	**Create append-only directory**	EASY	Nishan	*mkdir(appenddirname) on startup if appendonly=yes*
24	**Create append-only file**	EASY	Nishan	*Open/create .aof file with O_APPEND | O_WRONLY | O_CREAT*
25	**Create manifest file**	EASY	Nishan	*Write AOF manifest listing base and incremental files*
26	**Write a single command**	HARD	Nishan	*After each write cmd, serialise to RESP and fwrite to .aof*
27	**Write multiple commands**	MEDIUM	Nishan	*Buffer writes; flush on command boundary or fsync interval*
28	**Filter write commands**	EASY	Nishan	*Only log mutating commands (SET, DEL, LPUSH…); skip reads*
29	**Replay a single command**	HARD	Nishan	*On startup, open .aof, parse RESP, execute each command*
30	**Replay multiple commands**	MEDIUM	Nishan	*Handle partial writes / truncated file gracefully*
31	**Subscribe to a channel**	EASY	Divyanshi	*SUBSCRIBE ch → add client fd to channel subscriber set*
32	**Subscribe to multiple channels**	EASY	Divyanshi	*SUBSCRIBE ch1 ch2 → subscribe to each; confirm count*
33	**Enter subscribed mode**	MEDIUM	Divyanshi	*After SUBSCRIBE, client can only run SUBSCRIBE/UNSUBSCRIBE/PING*
34	**PING in subscribed mode**	EASY	Divyanshi	*Return *3 pong array instead of +PONG in subscribed mode*
35	**Publish a message**	EASY	Divyanshi	*PUBLISH ch msg → look up subscriber set; count receivers*
36	**Deliver messages**	HARD	Divyanshi	*Push *3 message array to every subscriber fd asynchronously*
37	**Unsubscribe**	MEDIUM	Divyanshi	*Remove fd from channel set; send confirmation; decrement count*

Person	Stages this week	Count
Aswin	*Create a stream (XADD), Validating entry IDs, Partially auto-generated IDs, Fully auto-generated IDs, Query entries (XRANGE), Query with - (min), Query with + (max), Query single stream (XREAD), Query multiple streams (XREAD), Blocking reads (XREAD BLOCK), Blocking reads without timeout, Blocking reads using $*	**12**
Ayurva	*The MULTI command, The EXEC command, Empty transaction, Queueing commands, Executing a transaction, The DISCARD command, Failures within transactions, Multiple transactions*	**8**
Nishan	*Default AOF options, AOF options from flags, Create append-only directory, Create append-only file, Create manifest file, Write a single command, Write multiple commands, Filter write commands, Replay a single command, Replay multiple commands*	**10**
Divyanshi	*Subscribe to a channel, Subscribe to multiple channels, Enter subscribed mode, PING in subscribed mode, Publish a message, Deliver messages, Unsubscribe*	**7**

WEEK 4 — Replication, Sorted Sets, Locking & Temporal

Jul 10 – Jul 16

Sprint Goal: Master-replica replication live, sorted sets done, optimistic locking done, Temporal WOW feature demoed.

	Stage / Command	Difficulty	Assigned To	What To Implement
1	**Configure listening port**	EASY	Amrit	*Accept --port flag; default 6379; bind to custom port*
2	**The INFO command**	EASY	Amrit	*Return server section: redis_version, role:master, etc.*
3	**INFO command on a replica**	MEDIUM	Amrit	*role:slave; master_host; master_port; master_repl_offset*
4	**Initial replication ID and offset**	EASY	Amrit	*Generate random 40-char master_replid; master_repl_offset=0*
5	**Send handshake (1/3)**	EASY	Amrit	*Replica sends PING to master; expect +PONG*
6	**Send handshake (2/3)**	EASY	Amrit	*Replica sends REPLCONF listening-port ; expect +OK*
7	**Send handshake (3/3)**	MEDIUM	Amrit	*Replica sends REPLCONF capa psync2; expect +OK*
8	**Receive handshake (1/2)**	EASY	Amrit	*Master receives REPLCONF; stores replica metadata*
9	**Receive handshake (2/2)**	EASY	Amrit	*Master receives PSYNC; responds with +FULLRESYNC replid 0*
10	**Empty RDB transfer**	EASY	Amrit	*Master sends hardcoded empty RDB binary after FULLRESYNC*
11	**Single-replica propagation**	MEDIUM	Amrit	*After each write on master, forward RESP command to replica fd*
12	**Multi-replica propagation**	HARD	Amrit	*Maintain list of replica fds; fan-out writes to all replicas*
13	**Command processing (replica)**	HARD	Amrit	*Replica executes propagated commands; no reply to master*
14	**ACKs with no commands**	EASY	Amrit	*REPLCONF GETACK * → replica replies REPLCONF ACK 0*
15	**ACKs with commands**	MEDIUM	Amrit	*Replica tracks byte offset; replies ACK*
16	**WAIT with no replicas**	MEDIUM	Amrit	*WAIT 0 0 → return 0 immediately (no replicas connected)*
17	**WAIT with no commands**	MEDIUM	Amrit	*WAIT n t when no writes → return replica count immediately*
18	**WAIT with multiple commands**	HARD	Amrit	*WAIT n t → send GETACK, collect ACKs, wait up to t ms*
19	**Create a sorted set (ZADD)**	EASY	Divyanshi	*Dual index: unordered_map scores + map*
20	**Add members**	MEDIUM	Divyanshi	*ZADD key score member; update both indexes; return added count*
21	**Retrieve member rank (ZRANK)**	MEDIUM	Divyanshi	*Count members with score < given member in the ordered map*
22	**List sorted set members (ZRANGE)**	EASY	Divyanshi	*Iterate ordered map from rank start to stop; return names*

	Stage / Command	Difficulty	Assigned To	What To Implement
23	**ZRANGE with negative indexes**	EASY	Divyanshi	*Negative rank wraps from end; resolve before iterating*
24	**Count sorted set members (ZCARD)**	EASY	Divyanshi	*Return size of scores map as RESP Integer*
25	**Retrieve member score (ZSCORE)**	MEDIUM	Divyanshi	*Lookup in scores map; return as Bulk String (float as string)*
26	**Remove a member (ZREM)**	EASY	Divyanshi	*Erase from both maps atomically*
27	**The WATCH command**	EASY	Ayurva	*WATCH key → register key in per-client watched_keys set*
28	**WATCH inside transaction**	EASY	Ayurva	*WATCH is valid before MULTI; keys already watched get re-watched*
29	**Tracking key modifications**	MEDIUM	Ayurva	*On any write: check if key is in any client's watched_keys*
30	**Watching multiple keys**	MEDIUM	Ayurva	*WATCH k1 k2 k3 → watch all; any modification triggers dirty flag*
31	**Watching missing keys**	EASY	Ayurva	*WATCH on non-existent key: mark dirty if key is created later*
32	**The UNWATCH command**	EASY	Ayurva	*UNWATCH → clear watched_keys; clear dirty flag*
33	**Unwatch on EXEC**	EASY	Ayurva	*After EXEC (success or dirty abort), auto-clear all watches*
34	**Unwatch on DISCARD**	EASY	Ayurva	*DISCARD also clears watches and dirty flag*
35	**TEMPORAL: GETVER command**	CUSTOM	Nishan	*GETVER key ts → binary search history vector for value at ts*
36	**TEMPORAL: HISTORY command**	CUSTOM	Nishan	*HISTORY key [n] → return last n versions as RESP array*
37	**TEMPORAL: SNAPSHOT command**	CUSTOM	Nishan	*SNAPSHOT ts → return entire store state at that timestamp*

Person	Stages this week	Count
Amrit	*Configure listening port, The INFO command, INFO command on a replica, Initial replication ID and offset, Send handshake (1/3), Send handshake (2/3), Send handshake (3/3), Receive handshake (1/2), Receive handshake (2/2), Empty RDB transfer, Single-replica propagation, Multi-replica propagation, Command processing (replica), ACKs with no commands, ACKs with commands, WAIT with no replicas, WAIT with no commands, WAIT with multiple commands*	18
Divyanshi	*Create a sorted set (ZADD), Add members, Retrieve member rank (ZRANK), List sorted set members (ZRANGE), ZRANGE with negative indexes, Count sorted set members (ZCARD), Retrieve member score (ZSCORE), Remove a member (ZREM)*	8
Ayurva	*The WATCH command, WATCH inside transaction, Tracking key modifications, Watching multiple keys, Watching missing keys, The UNWATCH command, Unwatch on EXEC, Unwatch on DISCARD*	8
Nishan	*TEMPORAL: GETVER command, TEMPORAL: HISTORY command, TEMPORAL: SNAPSHOT command*	3